UNDERACTUATED ROBOTICS

Algorithms for Walking, Running, Swimming, Flying, and Manipulation

Russ Tedrake

© Russ Tedrake, 2024
Last modified 2024-3-24.
How to cite these notes, use annotations, and give feedback.

Note: These are working notes used for a course being taught at MIT. They will be updated throughout the Spring 2024 semester. Lecture videos are available on YouTube.

Previous Chapter | Table of contents | Next Chapter

CHAPTER 20
Model-Free Policy Search

Reinforcement learning (RL) is a collection of algorithms for solving the same optimal control problem that we've focused on through the text, but the real gems from the RL literature are the algorithms for (almost) *black-box optimization* of stochastic optimal control problems. The idea of an algorithm that only has a "black-box" interface to the optimization problem means that it can obtain (potentially noisy) samples of the optimal cost via trial and error, but does not have access to the underlying model, and does not have direct access to complete gradient information.

This is a hard problem! In general we cannot expect RL algorithms to optimize as quickly as more structured optimization, and we can typically only guarantee convergence to a local optima at best. But the framework is extremely general, so it can be applied to problems that are inaccessible to any of the other algorithms we've examined so far. My favorite examples for reinforcement learning are control in complex fluid dynamics (e.g. [1]). These systems are often very difficult to model, or the model is so high-dimensional and complicated so that it's prohibitive for control design. In these problems, it might actually be faster to optimize via trial-and-error in physical experiments.

In this chapter we will examine one particular style of reinforcement learning that attempts to find good controllers by explicitly parameterizing a family of policies (e.g. via a parameter vector α), then searching directly for the parameters that optimize the long-term cost. For a stochastic optimal control problem with our favorite additive form of the objective, this might look like:

$$\min_\alpha E\left[\sum_{n=0}^{N} \ell(\mathbf{x}[n], \mathbf{u}[n])\right] \tag{1}$$

where the random variables are drawn from probability densities

$$\begin{gathered}\mathbf{x}[0] \sim p_0(\mathbf{x}),\\ \mathbf{x}[n] \sim p(\mathbf{x}[n]|\mathbf{x}[n-1], \mathbf{u}[n-1]),\\ \mathbf{u}[n] \sim p_\alpha(\mathbf{u}[n]|\mathbf{x}[n]).\end{gathered}$$

The last equation is a probabilistic representation of the control policy -- on each time-step actions $\mathbf{u}$ are drawn from a distribution that is conditioned on the current state, $\mathbf{x}$.

Of course, the controls community has investigated ideas like this, too, for instance under the umbrellas of *extremum-seeking control* and *iterative learning control*. I'll try to make connections whenever possible.

20.1 Policy Gradient Methods

One of the standard approaches to policy search in RL is to estimate the gradient of the expected long-term cost with respect to the policy parameters by evaluating some number of sample trajectories, then performing (stochastic) gradient descent. Many of these so-called "policy gradient" algorithms leverage a derivation called the *likelihood ratio method* that was perhaps first described in [2] then popularized in the the REINFORCE algorithm [3]. It is based on what looks like a trick with logarithms to estimate the gradient; I feel like this trick is often presented with an air of mystery about it. Let's try to make sure we understand it.

20.1.1 The Likelihood Ratio Method (aka REINFORCE)

Let's start with a simpler optimization over a stochastic function:

$$\min_{\alpha} E\left[g(\mathbf{x})\right] \text{ with } \mathbf{x} \sim p_\alpha(\mathbf{x})$$

I hope the notation is clear? $\mathbf{x}$ is a random vector, drawn from the distribution $p_\alpha(\mathbf{x})$, with the subscript indicating that the distribution depends on the parameter vector α. What is the gradient of this function? The REINFORCE derivation is

$$\begin{aligned}\frac{\partial}{\partial\alpha}E\left[g(\mathbf{x})\right] =& \frac{\partial}{\partial\alpha}\int d\mathbf{x}\; g(\mathbf{x})p_\alpha(\mathbf{x})\\ =& \int d\mathbf{x}\; g(\mathbf{x})\frac{\partial}{\partial\alpha}p_\alpha(\mathbf{x})\\ =& \int d\mathbf{x}\; g(\mathbf{x})p_\alpha(\mathbf{x})\frac{\partial}{\partial\alpha}\log p_\alpha(\mathbf{x})\\ =& E\left[g(\mathbf{x})\frac{\partial}{\partial\alpha}\log p_\alpha(\mathbf{x})\right].\end{aligned}$$

To achieve this, we used the derivative of the logarithm:

$$\begin{aligned}y &= \log u\\ \frac{\partial y}{\partial u} &= \frac{1}{u}\frac{\partial u}{\partial x}\end{aligned}$$

to write

$$\frac{\partial}{\partial\alpha}p_\alpha(\mathbf{x}) = p_\alpha(\mathbf{x})\frac{\partial}{\partial\alpha}\log p_\alpha(\mathbf{x}).$$

This suggests a simple Monte Carlo algorithm for estimating the policy gradient: draw N random samples $\mathbf{x}_i$ then estimate the gradient as

$$\frac{\partial}{\partial\alpha}E\left[g(\mathbf{x})\right] \approx \frac{1}{N}\sum_i g(\mathbf{x}_i)\frac{\partial}{\partial\alpha}\log p_\alpha(\mathbf{x}_i).$$

This trick is even more potent in the optimal control case. For a finite-horizon problem we have

$$\begin{aligned}\frac{\partial}{\partial\alpha}E\left[\sum_{n=0}^{N}\ell(\mathbf{x}[n],\mathbf{u}[n])\right] =& \int d\mathbf{x}[\cdot]d\mathbf{u}[\cdot]\left[\sum_{n=0}^{N}\ell(\mathbf{x}[n],\mathbf{u}[n])\right]\frac{\partial}{\partial\alpha}p_\alpha(\mathbf{x}[\cdot],\mathbf{u}[\cdot])\\ =& E\left[\left(\sum_{n=0}^{N}\ell(\mathbf{x}[n],\mathbf{u}[n])\right)\frac{\partial}{\partial\alpha}\log p_\alpha(\mathbf{x}[\cdot],\mathbf{u}[\cdot])\right],\end{aligned}$$

where $\mathbf{x}[\cdot]$ is shorthand for the entire trajectory $\mathbf{x}[0], \ldots, \mathbf{x}[N]$, and

$$p_\alpha(\mathbf{x}[\cdot], \mathbf{u}[\cdot]) = p_0(\mathbf{x}[0]) \left(\prod_{k=1}^{N} p\left(\mathbf{x}[k] \mid \mathbf{x}[k-1], \mathbf{u}[k-1]\right) \right) \left(\prod_{k=0}^{N} p_\alpha(\mathbf{u}[k] \mid \mathbf{x}[k]) \right).$$

Taking the $\log$ we have

$$\log p_\alpha\left(\mathbf{x}[\cdot], \mathbf{u}[\cdot]\right) = \log p_0(\mathbf{x}[0]) + \sum_{k=1}^{N} \log p(\mathbf{x}[k] \mid \mathbf{x}[k-1], \mathbf{u}[k-1]) + \sum_{k=0}^{N} \log p_\alpha(\mathbf{u}[k] \mid \mathbf{x}[k]).$$

Only the last terms depend on α, which yields

$$\begin{aligned} \frac{\partial}{\partial \alpha} E\left[\sum_{n=0}^{N} \ell(\mathbf{x}[n], \mathbf{u}[n])\right] &= E\left[\left(\sum_{n=0}^{N} \ell(\mathbf{x}[n], \mathbf{u}[n])\right)\left(\sum_{k=0}^{N} \frac{\partial}{\partial \alpha} \log p_\alpha\left(\mathbf{u}[k] \mid \mathbf{x}[k]\right)\right)\right] \\ &= E\left[\sum_{n=0}^{N}\left(\ell(\mathbf{x}[n], \mathbf{u}[n]) \sum_{k=0}^{n} \frac{\partial}{\partial \alpha} \log p_\alpha(\mathbf{u}[k] \mid \mathbf{x}[k])\right)\right], \end{aligned}$$

where the last step uses the fact that $E\left[\ell(\mathbf{x}[n], \mathbf{u}[n]) \frac{\partial}{\partial \alpha} \log p_\alpha(\mathbf{u}[k] \mid \mathbf{x}[k])\right] = 0$ for all $k > n$
.

This update should surprise you. It says that I can find the gradient of the long-term cost by taking only the gradient of the policy... but not the gradient of the plant, nor the cost! The intuition is that one can obtain the gradient by evaluating the policy along a number of (random) trajectory roll-outs of the closed-loop system, evaluating the cost on each, then increasing the probability in the policy of taking the actions correlated with lower long-term costs.

This derivation is often presented as **the** policy gradient derivation. The identity is certainly correct, but I'd prefer that you think of this as just one way to obtain the policy gradient. It's a particularly clever one in that it uses exactly the information that we have available in reinforcement learning -- we have access to the instantaneous costs, $\ell(\mathbf{x}[n], \mathbf{u}[n])$, and to the policy -- so are allowed to take gradients of the policy -- but does not require any understanding whatsoever of the plant model. Although it's clever, it is not particularly efficient -- the Monte Carlo approximations of the expected value have a high variance, so many samples are required for an accurate estimate. Other derivations are possible, some even simpler, and others that **do** make use of the plant gradients if you have access to them, and these will perform differently in the finite-sample approximations.

For the remainder of this section, I'd like to dig in and try to understand the nature of the stochastic update a little better, to help you understand what I mean.

20.1.2 Sample efficiency

Let's take a step back and think more generally about how one can use gradient descent in black-box (unconstrained) optimization. Imagine you have a simple optimization problem:

$$\min_\alpha g(\alpha),$$

and you can directly evaluate $g()$, but not $\frac{\partial g}{\partial \alpha}$. How can you perform gradient descent?

A standard technique for estimating the gradients, in lieu of analytical gradient information, is the method of finite differences[4]. The finite differences approach to estimating the gradient involves making the same small perturbation, ϵ to the input parameters in every dimension independently, and using:

$$\frac{\partial g}{\partial \alpha_i} \approx \frac{g(\alpha + \epsilon_i) - g(\alpha)}{\epsilon},$$

where $\epsilon_\mathbf{i}$ is the column vector with ϵ in the ith row, and zeros everywhere else. Finite difference methods can be computationally very expensive, requiring $n+1$ evaluations of the function for every gradient step, where n is the length of the input vector.

What if each function evaluation is expensive? Perhaps it means picking up a physical robot and running it for 10 seconds for each evaluation of $g()$. Suddenly there is a premium on trying to optimize the cost function with the fewest number of evaluations on $g()$. This is the name of the game in reinforcement learning -- it's often called RL's *sample complexity*. Can we perform gradient descent using less evaluations?

20.1.3 Stochastic Gradient Descent

This leads us to the question of doing approximate gradient descent, or "stochastic" gradient descent. Thinking of the cost landscape as a Lyapunov function[†], then any update that moves downhill on each step will eventually reach the optimal. More generally, any update that moves downhill on average will eventually reach a minimum... and sometimes "stochastic" gradient descent updates that occassionally go uphill but go downhill on average can even have desirable properties like bouncing out of small local minima. The figure below gives some graphical intuition; for a formal treatment of stochastic gradient methods, see e.g. [5].

[†] the Lyapunov functi $V(\alpha) = g(\alpha) - g(\alpha^*)$ is commonly used in convergence/stability analysis of optimizati algorithms.

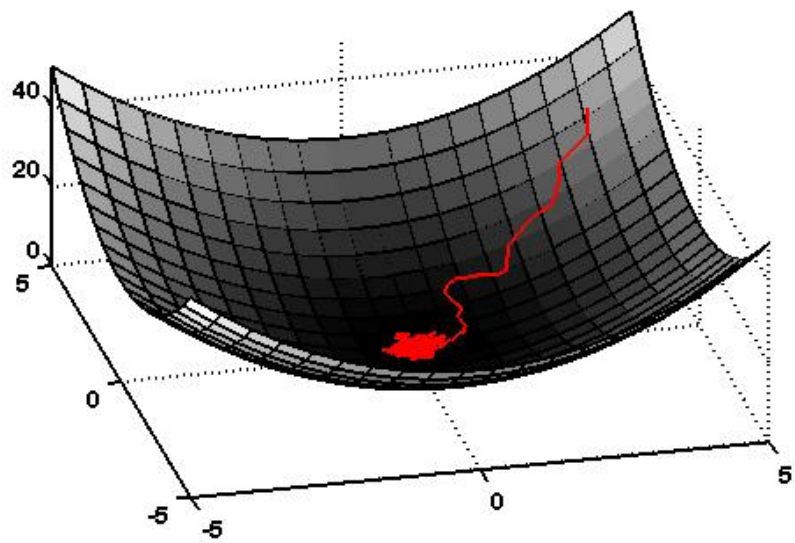

Figure 20.1 - Stochastic gradient descent of a quadratic cost function .

20.1.4 The Weight Pertubation Algorithm

Instead of sampling each dimension independently, consider making a single small random change, β, to the parameter vector, α. Now consider the update of the form:

$$\Delta\alpha = -\eta[g(\alpha + \beta) - g(\alpha)]\beta.$$

The intuition here is that if $g(\alpha + \beta) < g(\alpha)$, then β was a good change and we'll move in the direction of β. If the cost increased, then we'll move in the opposite direction. Assuming the function is smooth and β is small, then we will always move downhill on the Lyapunov function.

Even more interesting, on average, the update is actually in the direction of the true gradient. To see this, again we can assume the function is locally smooth and β is small to write the Taylor expansion:

$$g(\alpha+\beta) \approx g(\alpha) + \frac{\partial g}{\partial \alpha}\beta.$$

Now we have

$$\begin{aligned}\Delta\alpha &\approx -\eta\left[\frac{\partial g}{\partial \alpha}\beta\right]\beta = -\eta\beta\beta^T\frac{\partial g}{\partial \alpha}^T \\ E\left[\Delta\alpha\right] &\approx -\eta E\left[\beta\beta^T\right]\frac{\partial g}{\partial \alpha}^T\end{aligned}$$

If we select each element of β independently from a distribution with zero mean and variance σ_β^2, or $E\left[\beta_i\right] = 0, E\left[\beta_i\beta_j\right] = \sigma_\beta^2\delta_{ij}$, then we have

$$E\left[\Delta\alpha\right] \approx -\eta\sigma_\beta^2\frac{\partial g}{\partial \alpha}^T.$$

Note that the distribution $p_\alpha(\mathbf{x})$ need not be Gaussian, but it is the variance of the distribution which determines the scaling on the gradient.

20.1.5 Weight Perturbation with an Estimated Baseline

The weight perturbation update above requires us to evaluate the function g twice for every update of the parameters. This is low compared to the method of finite differences, but let us see if we can do better. What if, instead of evaluating the function twice per update [$g(\alpha+\beta)$ and $g(\alpha)$], we replace the evaluation of $g(\alpha)$ with an estimator, $b = \hat{g}(\alpha)$, obtained from previous trials? In other words, consider an update of the form:

$$\Delta\alpha = -\frac{\eta}{\sigma_\beta^2}[g(\alpha+\beta) - b]\beta \tag{2}$$

The estimator can take any of a number of forms, but perhaps the simplest is based on the update (after the nth trial):

$$b[n+1] = \gamma g[n] + (1-\gamma)b[n], \quad b[0] = 0, 0 \le \gamma \le 1,$$

where γ parameterizes the moving average. Let's compute the expected value of the update, using the same Taylor expansion that we used above:

$$\begin{aligned}E[\Delta\alpha] &= -\frac{\eta}{\sigma_\beta^2}E\left[[g(\alpha) + \frac{\partial g}{\partial \alpha}\beta - b]\beta\right] \\ &= -\frac{\eta}{\sigma_\beta^2}E\left[[g(\alpha) - b]\beta\right] - \frac{\eta}{\sigma_\beta^2}E\left[\beta\beta^T\right]\frac{\partial g}{\partial \alpha}^T \\ &= -\eta\frac{\partial g}{\partial \alpha}^T\end{aligned}$$

In other words, the baseline does not effect our basic result - that the expected update is in the direction of the gradient. Note that this computation works for any baseline estimator which is uncorrelated with β, as it should be if the estimator is a function of the performance in previous trials.

Although using an estimated baseline doesn't effect the average update, it can have a dramatic effect on the performance of the algorithm. As we will see in a later section, if the evaluation of g is stochastic, then the update with a baseline estimator can actually outperform an update using straight function evaluations.

Let's take the extreme case of $b = 0$. This seems like a very bad idea... on each step

we will move in the direction of every random perturbation, but we will move more or less in that direction based on the evaluated cost. In average, we will still move in the direction of the true gradient, but only because we will eventually move more downhill than we move uphill. It feels very naive.

20.1.6 REINFORCE w/ additive Gaussian noise

Now let's consider the simple form of the REINFORCE update:

$$\frac{\partial}{\partial \alpha} E\left[g(\mathbf{x})\right] = E\left[g(\mathbf{x})\frac{\partial}{\partial \alpha}\log p_\alpha(\mathbf{x})\right].$$

It turns out that weight perturbation is a REINFORCE algorithm. To see this, take $\mathbf{x} = \alpha + \beta$, $\beta \in N(0, \sigma^2)$, to have

$$p_\alpha(\mathbf{x}) = \frac{1}{(2\pi\sigma^2)^N} e^{\frac{-(\mathbf{x}-\alpha)^T(\mathbf{x}-\alpha)}{2\sigma^2}}$$
$$\log p_\alpha(\mathbf{x}) = \frac{-(\mathbf{x}-\alpha)^T(\mathbf{x}-\alpha)}{2\sigma^2} + \text{terms that don't} \quad \text{depend on } \alpha$$
$$\frac{\partial}{\partial \alpha}\log p_\alpha(\mathbf{x}) = \frac{1}{\sigma^2}(\alpha - \mathbf{x})^T = \frac{1}{\sigma^2}\beta^T.$$

If we use only a single trial for each Monte-Carlo evaluation, then the REINFORCE update is

$$\Delta\alpha = -\frac{\eta}{\sigma^2} g(\alpha + \beta)\beta,$$

which is precisely the weight perturbation update (the crazy version with $b = 0$ discussed above). Although it will move in the direction of the gradient on average, it can be highly inefficient. In practice, people use many more than one sample to estimate the poilicy gradient.

20.1.7 Summary

The policy gradient "trick" from REINFORCE using log probabilities provides one means of estimating the true policy gradient. It is not the only way to obtain the policy gradient... in fact the trivial weight-perturbation update obtains the same gradient for the case of a policy with a mean linear in α and a fixed diagonal covariance. It's cleverness lies in the fact that it makes use of the information we do have (instantaneous cost values and the gradients of the policy), and that it provides an unbiased estimate of the gradient (note that taking the gradients of a model that is only slightly wrong might not have this virtue). But its inefficiency comes from having a potentially very high variance. Variance reduction for policy gradient, often through baseline estimation, continues to be an active area of research.

20.2 SAMPLE PERFORMANCE VIA THE SIGNAL-TO-NOISE RATIO.

20.2.1 Performance of Weight Perturbation

The simplicity of the REINFORCE / weight perturbation updates makes it tempting to apply them to problems of arbitrary complexity. But a major concern for the algorithm is its performance - although we have shown that the update is in the direction of the true gradient *on average*, it may still require a prohibitive number of computations to obtain a local minima.

In this section, we will investigate the performance of the weight perturbation algorithm by investigating its *signal-to-noise* ratio (SNR). This idea was explored in

[1] -- my goal is just to give you a taste here. The SNR is the ratio of the power in the signal (here desired update in the direction of the true gradient) and the power in the noise (the remaining component of the update, so that $\Delta\alpha = -\eta \frac{\partial g}{\partial \alpha}^T + \text{noise}$)[†]

$$\text{SNR} = \frac{-\eta \frac{\partial g}{\partial \alpha}^{T\ 2}}{E\left[\ \Delta\alpha + \eta \frac{\partial g}{\partial \alpha}^{T\ 2}\right]}.$$

In the special case of the unbiased update, the equation reduces to:

$$\text{SNR} = \frac{E[\Delta\alpha]^T E\left[\Delta\alpha\right]}{E\left[(\Delta\alpha)^T(\Delta\alpha)\right] - E[\Delta\alpha]^T E\left[\Delta\alpha\right]}.$$

For the weight perturbation update we have:

$$E[\Delta\alpha]^T E\left[\Delta\alpha\right] = \eta^2 \frac{\partial g}{\partial \alpha} \frac{\partial g}{\partial \alpha}^T = \eta^2 \sum_{i=1}^{N} \left(\frac{\partial g}{\partial \alpha_i}\right)^2,$$

$$\begin{aligned}
E\left[(\Delta\alpha)^T(\Delta\alpha)\right] =& \frac{\eta^2}{\sigma_\beta^4} E\left[\left[g(\alpha+\beta) - g(\alpha)\right]^2 \beta^T\beta\right] \\
\approx& \frac{\eta^2}{\sigma_\beta^4} E\left[\left[\frac{\partial g}{\partial \alpha}\beta\right]^2 \beta^T\beta\right] \\
=& \frac{\eta^2}{\sigma_\beta^4} E\left[\left(\sum_i \frac{\partial g}{\partial \alpha_i}\beta_i\right)\left(\sum_j \frac{\partial g}{\partial \alpha_j}\beta_j\right)\left(\sum_k \beta_k^2\right)\right] \\
=& \frac{\eta^2}{\sigma_\beta^4} E\left[\sum_i \frac{\partial g}{\partial \alpha_i}\beta_i \sum_j \frac{\partial g}{\partial \alpha_j}\beta_j \sum_k \beta_k^2\right] \\
=& \frac{\eta^2}{\sigma_\beta^4} \sum_{i,j,k} \frac{\partial g}{\partial \alpha_i}\frac{\partial g}{\partial \alpha_j} E\left[\beta_i\beta_j\beta_k^2\right]
\end{aligned}$$

$$E\left[\beta_i\beta_j\beta_k^2\right] = \begin{cases} 0 & i \neq j \\ \sigma_\beta^4 & i = j \neq k \\ \mu_4(\beta) & i = j = k \end{cases}$$

$$= \eta^2(N-1)\sum_i \left(\frac{\partial g}{\partial \alpha_i}\right)^2 + \frac{\eta^2\mu_4(z)}{\sigma_\beta^4}\sum_i \left(\frac{\partial g}{\partial \alpha_i}\right)^2$$

where $\mu_n(z)$ is the nth central moment of z:

$$\mu_n(z) = E\left[(z - E\left[z\right])^n\right].$$

Putting it all together, we have:

$$\text{SNR} = \frac{1}{N - 2 + \frac{\mu_4(\beta_i)}{\sigma_\beta^4}}.$$

Example 20.1 (Signal-to-noise ratio for additive Gaussian noise)

For β_i drawn from a Gaussian distribution, we have $\mu_1 = 0, \mu_2 = \sigma_\beta^2, \mu_3 = 0, \mu_4 = 3\sigma_\beta^4$, simplifying the above expression to:

$$\text{SNR} = \frac{1}{N+1}.$$

[†] SNR could alternatively be defin as the ratio of the power of the compone of the update in the direction of the signal and the component orthogonal to the sign (does not penalize the magnitudes of the updates):

$$\frac{E[a^2]}{E\left[|\Delta\alpha - a\mathbf{v}|^2\right]},$$

where $\mathbf{v} = \frac{\frac{\partial g}{\partial \alpha}^T}{\frac{\partial g}{\partial \alpha}^T}$, $a = \Delta\alpha \cdot \mathbf{v}$. This form i probably a better definition, but is more difficult to work with.

Example 20.2 (Signal-to-noise ratio for additive uniform noise)

For β_i drawn from a uniform distribution over [-a,a], we have $\mu_1 = 0, \mu_2 = \frac{a^2}{3} = \sigma_\beta^2, \mu_3 = 0, \mu_4 = \frac{a^4}{5} = \frac{9}{5}\sigma_\beta^4$, simplifying the above to:

$$\text{SNR} = \frac{1}{N - \frac{1}{5}}.$$

Performance calculations using the SNR can be used to design the parameters of the algorithm in practice. For example, based on these results it is apparent that noise added through a uniform distribution yields better gradient estimates than the Gaussian noise case for very small N, but that these differences are neglibile for large N.

Similar calculations can yield insight into picking the size of the additive noise (scaled by σ_β). The calculations in this section seem to imply that larger σ_β can only reduce the variance, overcoming errors or noise in the baseline estimator, $\tilde{b}$; this is a short-coming of our first-order Taylor expansion. If the cost function is not linear in the parameters, then an examination of higher-order terms reveals that large σ_β can increase the SNR. The derivation with a second-order Taylor expansion is left for an exercise.

References

1. John W. Roberts, "Motor Learning on a Heaving Plate via Improved-{SNR} Algorithms", , February, 2009. [link]

2. Peter W. Glynn, "Likelihood ratio gradient estimation for stochastic systems", *Communications of the ACM*, vol. 33, no. 10, pp. 75--84, oct, 1990.

3. R.J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning", *Machine Learning*, vol. 8, pp. 229-256, 1992.

4. William H. Press and Saul A. Teukolsky and William T. Vetterling and Brian P. Flannery, "Numerical Recipes in C: The Art of Scientific Computing", Cambridge University Press , 1992.

5. Dimitri P Bertsekas, "Nonlinear programming", Athena scientific Belmont , 1999.

© Russ Tedrake, 2024

Underactuated Robotics

Algorithms for Walking, Running, Swimming, Flying, and Manipulation

Russ Tedrake

© Russ Tedrake, 2024
Last modified 2024-3-24.
How to cite these notes, use annotations, and give feedback.

Note: These are working notes used for a course being taught at MIT. They will be updated throughout the Spring 2024 semester. Lecture videos are available on YouTube.

Previous Chapter | Table of contents | Next Chapter

Appendix A
Drake

Drake is the main software toolbox that we use for this text, which in fact originated in a large part due to the MIT underactuated course. The Drake website is the main source for information and documentation. The goal of this chapter is to provide any additional information that can help you get up and running with the examples and exercises provided in these notes.

A.1 Pydrake

Drake is primarily a C++ library, with rigorous coding standards and a maturity level intended to support even professional applications in industry. In order to provide a gentler introduction, and to facilitate rapid prototyping, I've written these notes exclusively in python, using Drake's python bindings (pydrake). These bindings are less mature than the C++ backend; your feedback (and even contributions) are very welcome. It is still improving rapidly.

In particular, while the C++ API documentation is excellent, the autogenerated python docs are still a work in progress. I currently recommend using the C++ documentation to find what you need, then checking the Python documentation only if you need to understand how the class or method is spelled in pydrake.

There are also a number of tutorials in Drake that can help you get started.

A.2 Online Jupyter Notebooks

I will provide nearly all examples and exercises in the form of a Jupyter Notebook so that we can leverage the fantastic and relatively recent availability of (free) cloud resources.

A.2.1 Running on Deepnote

We'll use Deepnote as the primary platform for the course. After following any of the links from the chapters, you should:

1. Log in (the free account will be sufficient for this class)
2. "Duplicate" the document. Icon is in the top right next to Login.
3. Run all of the cells (can use the "Run notebook" icon just above this cell)
4. Many of the notebooks use MeshCat for interactive visualizations. Click on the url printed just below "StartMeshcat" (often the second code cell of the notebook) to see the MeshCat window.

A.2.2 Running on Google Colab

As of now, Drake no longer supports Google Colab, which is stuck on Ubuntu 18.04 and Python 3.7. We will try to support it again if/when they upgrade.

A.2.3 Enabling licensed solvers

You can enabled more powerful solvers for `MathematicalProgram` if you have a license (most are free for academics). Please see this tutorial for instructions.

A.3 Running on your own machine

As you get more advanced, you will likely want to run (and extend) these examples on your own machine. On platforms/configurations that Drake supports (the latest two releases of Mac and Ubuntu using the system default python versions). The simplest installation is via `pip`; I would typically recommend using a virtual environment:

Copy code

```
python3 -m venv venv
source venv/bin/activate
pip3 install underactuated --extra-index-url https://drake-packages.csail.mit.edu/whl/night
```

Then you can download notebooks directly from Deepnote and run them locally (or develop your own notebooks).

The Drake website also has a number of alternative installation options, including precompiled binaries and Docker instances.

If you want to download all of the notebooks at once, you can `git clone` the course notes repository. You'll likely want to start from the repository's root directory. Then launch your notebook with:

Copy code

```
jupyter notebook
```

The examples for each chapter that has them will be in a .ipynb file right alongside the chapter's html file, and the notebook exercises are all located in the `exercises` subdirectory.

A.4 Getting help

If you have trouble with Drake, please follow the advice here. If you have trouble with the underactuated repo, you can check for known issues (and potentially file a new one) here.

© Russ Tedrake, 2024